Propuesta de Proyecto TFM: Copiloto de Posventa Autónomo y Gestión Inteligente de Devoluciones (ReturnWorkflow)

1. Descripción de la propuesta

Este proyecto propone el diseño y desarrollo de "ReFlow", un sistema automatizado para la gestión de devoluciones y soporte posventa en entornos de comercio electrónico. La solución se basa en una arquitectura de agentes impulsada por Inteligencia Artificial Generativa y desarrollada íntegramente en Python.

Técnicamente, el sistema empleará **LangChain** para la orquestación de un flujo multi-agente, interactuando con bases de datos relacionales (PostgreSQL) y vectoriales (pgvector en Supabase) mediante el protocolo **MCP (Model Context Protocol)**. El backend, construido sobre **FastAPI**, expondrá los servicios necesarios para que una interfaz de usuario en **Streamlit** permita a los clientes gestionar sus solicitudes de forma conversacional.

2. Motivaciones de la misma

Las devoluciones y consultas sobre el estado de los pedidos representan más del 60% del volumen de interacciones en el servicio de atención al cliente de un e-commerce. La validación manual del cumplimiento de las políticas de devolución y la posterior tramitación logística consumen una cantidad excesiva de tiempo humano.

La motivación principal de este TFM es aplicar técnicas avanzadas de IA Generativa (como RAG y orquestación de agentes) para resolver un problema real y costoso en el sector retail. Al automatizar la evaluación de elegibilidad y la generación de etiquetas de envío, se busca optimizar los recursos, mejorar la experiencia del cliente (ofreciendo respuestas inmediatas 24/7) y demostrar la capacidad técnica para diseñar e implementar un producto de software robusto, escalable y orientado al backend.

3. Objetivos finales que se persiguen

- **Desarrollar una solución integral en Python:** Implementar la lógica core del sistema, incluyendo la orquestación de LLMs, la conexión a bases de datos y la exposición de APIs, utilizando exclusivamente herramientas basadas en código y frameworks

profesionales.

- **Implementar un sistema RAG eficiente:** Integrar una base de datos vectorial (Supabase/pgvector) para recuperar y aplicar con precisión las políticas de devolución específicas de cada producto o categoría, minimizando las alucinaciones del modelo.
- **Diseñar una arquitectura multi-agente robusta:** Utilizar LangChain para separar las responsabilidades (triaje, evaluación de políticas y resolución), garantizando un flujo de ejecución predecible y seguro.
- **Incorporar un mecanismo "Human-in-the-Loop" (HITL):** Implementar controles de seguridad que pausen el proceso automático para devoluciones de alto valor (ej. > 100€), derivándolas a un panel de supervisión humana para su aprobación manual.
- **Construir un Producto Mínimo Viable (MVP) funcional y reproducible:** Entregar una aplicación ejecutable que demuestre el flujo completo, desde la interacción del cliente hasta la generación (simulada) de la etiqueta de transporte, asegurando la escalabilidad de la arquitectura.
- **Monitorización y análisis de viabilidad (ROI)**: Evaluar la eficiencia del sistema mediante métricas de latencia, precisión (usando frameworks como LangSmith) y estimación de costes de inferencia (tokens) para garantizar la viabilidad técnica y económica de la arquitectura de cara a su uso en un entorno productivo.
- **Asegurar la reproducibilidad y seguridad:** Diseñar un entorno de despliegue Reproducible en AWS.

4. Alcance y Viabilidad (MVP)

Para garantizar el éxito del proyecto en el tiempo disponible, la propuesta se acota al desarrollo de un Producto Mínimo Viable (MVP) o Prueba de Concepto. El enfoque técnico se concentrará en la calidad, escalabilidad y diseño de la lógica backend y de IA. La capa de presentación se mantendrá ligera, utilizando **Streamlit** para construir un frontend simple pero efectivo que permita validar la interacción con la API.